



## Ayudantía 05 2026-2

### ZYBOZ7 – Protocolo AXI y RGB driver

Ayudante: Ernesto Ferrante – [ernesto.ferrante@uc.cl](mailto:ernesto.ferrante@uc.cl)

Prof. Michel Rozas Fernández – [mvrozas@uc.cl](mailto:mvrozas@uc.cl)

---

## 1. Objetivo de la Ayudantía

- Ejercitar protocolo AXI.
- Introducir creación de IPCores con puerto AXI.
- Manejar LED RGB ZyboZ7.
- Generación de señal PWM.

## 2. Actividades Previas a la Ayudantía

En el vídeo asociado a nuestra ayudantía 05 se creó un IPCore con puerto AXI esclavo *RGB\_Driver*, el cual recibe 4 constantes de comparación desde un AXI Traffic Generator para ser comparada con un contador diente de sierra y de esta manera, a través del ciclo de trabajo de la señal PWM producto de la comparación, variar la intensidad lumínica del led RGB. Se cargan los archivos *.COE* en el IPCore ATG para la correcta transacción de datos, y se guardan en 4 registros esclavos del IPCore *RGB\_Driver*. Se valida el protocolo por medio de la incorporación de un bloque ILA, logrando visualizar las principales señales de control e información utilizadas para una transacción del tipo AXI lite.

- Cree una IPCore con puerto AXI desde la herramienta *tools* → *CreateAndPackageNewIP* en la barra superior de Vivado.
- Se abrirá un proyecto nuevo asociado al IPCore, incluya la fuente de menor jerarquía el archivo *.VHD RGB\_driver\_v1\_0\_S00\_AXI*, en la fuente de mayor jerarquía incluya *RGB\_driver\_v1\_0*. Finalmente, empaquete el IPCore e incorpórelo desde el diagrama de bloques *add IP*.

- Incorpore un AXI Traffic Generator, configurado en Test Mode, protocolo AXI lite. Añada los archivo *.COE* de control, dirección, datos y mascara disponibles en el repositorio.
- Incorpore un bloque ILA *MonitorType* AXI para medir y validar la correcta transacción de datos a través de AXI entre el ATG y el *RGB\_driver*.
- Finalmente, cree el HDL Wrapper y genere el Bitstream.

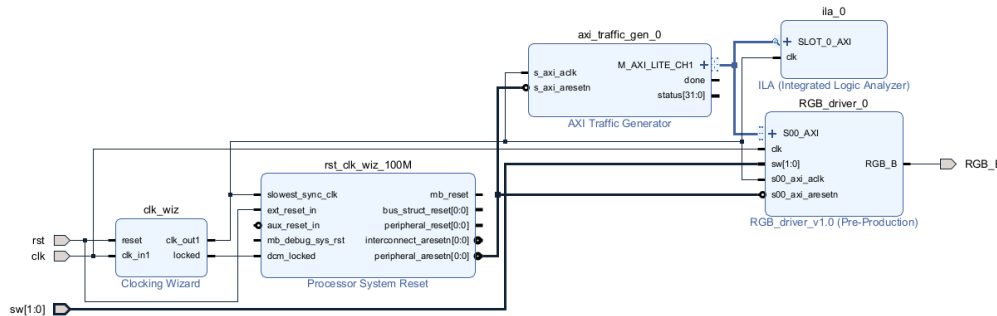


Figura 1: Block Design proyecto ayudantía 05 Vivado.

### 3. Actividades Durante la Ayudantía

Durante la ayudantía deberá modificar el IPCore para generar los 16 colores de la tabla de la Fig. 2 en el LED RGB, seleccionables mediante los 4 switches de la ZyboZ7. Para esto considere más salidas en el *RGB\_driver*, específicamente, el LED rojo y el LED verde. Cambie las intensidades de color de cada LED, para esto considere enviar más datos desde el ATG en función de los colores que desea generar y utilice más registros esclavos donde almacenarlos.

- Modifique el IPCore para exponer las tres salidas del LED RGB y ampliar la entrada de switches a 4 bits. Recuerde propagar los puertos nuevos al archivo de mayor jerarquía y re-empaquetar el IPCore antes de incorporarlo al Block Design.
- Amplíe la cantidad de registros esclavos del IPCore. Observando la tabla de la Fig. 2, note que los 16 colores se construyen a partir de un número reducido de niveles de intensidad distintos; determine cuántos son y dimensione los registros en consecuencia.
- Modifique sus archivos *.COE* (datos, dirección, control y máscara) para cargar dichos niveles de intensidad en los registros esclavos. Verifique que los cuatro archivos tengan la misma cantidad de transacciones y que las direcciones correspondan al mapa asignado en el *Address Editor*.
- Cree un código tipo *Look-Up* Table que reciba de entrada los 4 switches y seleccione, para cada canal del LED, el registro esclavo que contiene la intensidad correspondiente al color solicitado.

- Incorpore un mecanismo de parpadeo (*blink*) que encienda y apague el color seleccionado de forma periódica, sin alterar el tono. Considere que el reloj del sistema es de 125 MHz.
- Valide mediante el bloque ILA que las transacciones AXI hacia el  $RGB_{driver}$  se realicen correctamente, capturando las señales de dirección, dato y control.

A modo de ejemplo;

| Color Chart | R   | G   | B   | Color Name   |
|-------------|-----|-----|-----|--------------|
| ■ ■ ■       | 0   | 0   | 0   | Black        |
| ■ ■ ■       | 255 | 255 | 255 | White        |
| ■ ■ ■       | 224 | 224 | 224 | Light Gray   |
| ■ ■ ■       | 128 | 128 | 128 | Gray         |
| ■ ■ ■       | 64  | 64  | 64  | Dark Gray    |
| ■ ■ ■       | 255 | 0   | 0   | Red          |
| ■ ■ ■       | 255 | 96  | 208 | Pink         |
| ■ ■ ■       | 160 | 32  | 255 | Purple       |
| ■ ■ ■       | 80  | 208 | 255 | Light Blue   |
| ■ ■ ■       | 0   | 32  | 255 | Blue         |
| ■ ■ ■       | 96  | 255 | 128 | Yellow-Green |
| ■ ■ ■       | 0   | 192 | 0   | Green        |
| ■ ■ ■       | 255 | 224 | 32  | Yellow       |
| ■ ■ ■       | 255 | 160 | 16  | Orange       |
| ■ ■ ■       | 160 | 128 | 96  | Brown        |
| ■ ■ ■       | 255 | 208 | 160 | Pale Pink    |

Figura 2: Tabla valores RGB.

Suponga que quiere generar el color morado, teniendo en consideración que el máximo de luminosidad en cada LED es de 255 (Ciclo de trabajo unitario). Necesitará aplicar, según la Fig. 2, un ciclo de trabajo  $\alpha_{rojo} = \frac{160}{255}$ ,  $\alpha_{verde} = \frac{32}{255}$  y  $\alpha_{azul} = \frac{255}{255}$  en cada LED correspondiente. Tenga en cuenta la amplitud del contador que estamos utilizando para mandar el dato a través de AXI al IPCore que generará el ciclo de trabajo necesario para el color que tiene que generar. Note que los valores de la tabla están en el rango  $[0, 255]$ , mientras que el contador del PWM alcanza un valor máximo distinto: deberá escalar cada nivel antes de escribirlo en el archivo *.COE*. Definiendo las intensidades que necesita, deberá a través de los Switches generar un color. También le puede ser útil;

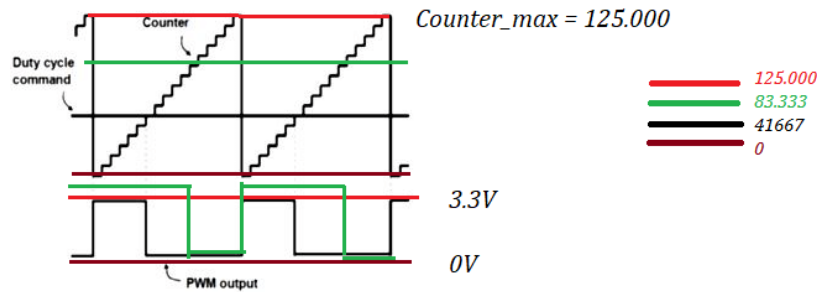


Figura 3: PWM.

El Ciclo de Trabajo se considera como;

$$\alpha = \frac{t_{on}}{T}$$

Dónde  $t_{on}$  es el tiempo que pasa la señal en estado HIGH (3.3 V) y  $T$  es el periodo de PWM.

**El tiempo de *blink* puede ser de 1 o de 2 segundos. Si tiene un grupo de estudiantes al lado, pónganse de acuerdo para trabajar con tiempos de *blink* distintos.**